Repositorio del proyecto: https://github.com/donmatthiuz/RL/tree/proyecto

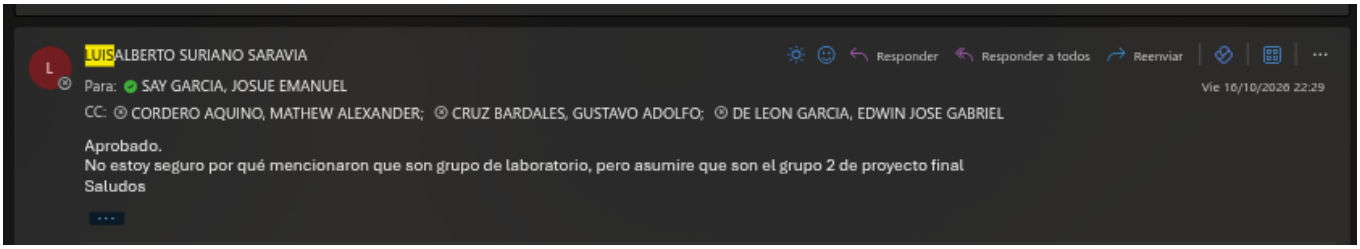
EVIDENCIA DOCUMENTAL

Constancia de entrega y aprobación

Propuesta de proyecto: Aprendizaje por Refuerzo en Polymarket

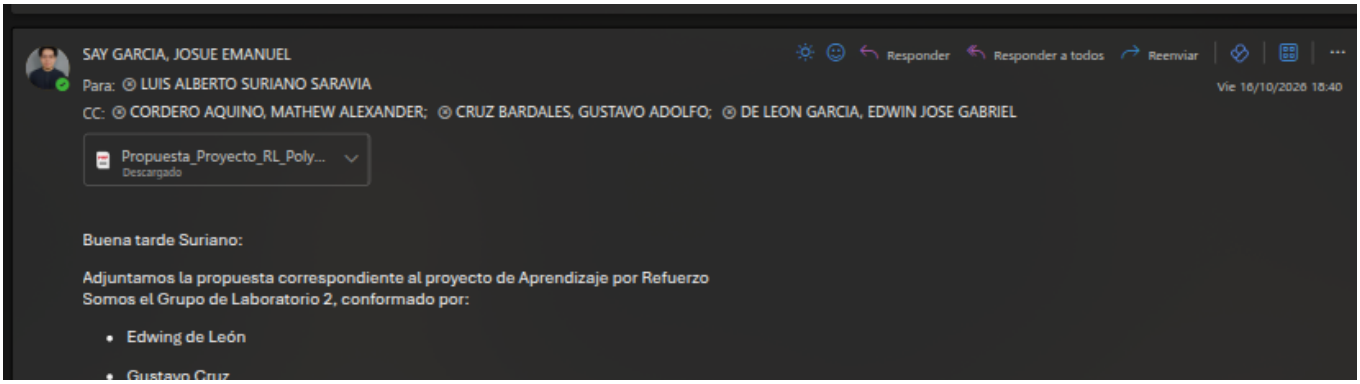
Se incorporan al inicio las capturas aportadas como evidencia de entrega oportuna de la propuesta original. En ellas constan el envío del **16 de octubre de 2026 a las 18:40** y la aprobación del profesor ese mismo día a las **22:29**.

1. Aprobación del profesor



Luis Alberto Suriano Saravia · 16/10/2026, 22:29. El mensaje confirma: «Aprobado». También indica que asume que el equipo corresponde al **grupo 2 del proyecto final**.

2. Envío de la propuesta



Jose Emanuel Say García a Luis Alberto Suriano Saravia · 16/10/2026, 18:40. La captura muestra el correo de envío, el archivo adjunto y la referencia al proyecto de Aprendizaje por Refuerzo.

> **Actualización de la propuesta: 2 de noviembre de 2026.**
> Las páginas siguientes desarrollan la propuesta original e incorporan las decisiones, el código y los resultados registrados en el repositorio entre el 30 de octubre y el 2 de noviembre. Las capturas se conservan sin modificaciones.

PROPUESTA ACTUALIZADA · 2 DE NOVIEMBRE DE 2026

PolyRL: aprendizaje por refuerzo en Polymarket

Datos históricos, decisiones secuenciales y operaciones con saldo ficticio.

Idea y objetivo

Construir un agente que observe un mercado binario histórico y aprenda cuándo **comprar YES, comprar NO, vender o esperar**. El objetivo es comprobar si su política supera a estrategias simples en mercados que no se utilizaron para entrenarlo.

Polymarket funciona como fuente de datos. El entrenamiento y la evaluación se hacen en un simulador con **saldo ficticio**. El proyecto integra Deep Reinforcement Learning, datos reales, modelado de un entorno y evaluación estadística.

Cómo funciona

- **1. Datos:** 12 294 mercados resueltos *Bitcoin Up or Down* horarios, de mayo de 2025 a octubre de 2026, con precio por minuto alineado con el spot de Binance. La extensión a ETH (P2) no se hizo.
- **2. Simulación:** cada mercado es un episodio de 60 decisiones, con una posición a la vez de 20 USDC, costos y liquidación al conocerse el resultado.
- **3. Aprendizaje:** PPO y DQN se entrenaron interactuando con el simulador; CQL y BC, con un conjunto fijo de transiciones de train.
- **4. Comparación:** todas las políticas se jugaron en los mismos mercados de val y, una sola vez, en los de test, midiendo retorno, riesgo, operaciones y costos.

Técnicas utilizadas

Técnica	Aplicación
PPO	Agente principal; actor-crítico de stable-baselines3, 5 semillas con la mejor configuración de val.
DQN	Agente de comparación; estima el valor de las cuatro acciones discretas. 5 semillas.
Offline RL	Discrete CQL de d3rlpy y Behavior Cloning como referencia, ambos sobre transiciones fijas de train.
Backtesting y recompensa	Reproducción histórica y ablación de la recompensa: ganó el PnL a precio medio con penalización de drawdown.

> **Precisión conceptual incorporada:** usar precios históricos en un simulador no convierte por sí solo a PPO o DQN en Offline RL. Si el agente interactúa con el entorno, el entrenamiento es online. La variante offline aprende de transiciones ya recolectadas.

Resultado a la fecha

La demo local en Streamlit ya reproduce un mercado paso a paso con la acción, la posición y el saldo, y compara políticas con su PnL acumulado. En test ningún agente superó a no operar; el informe explicará este resultado negativo y sus causas.

DISEÑO TÉCNICO

Datos, entorno y recompensa

Lo previsto en la propuesta, ya implementado y probado en el paquete polym4v.

Módulo	Estado y contenido
Datos	**Implementado.** Crudo archivado con hashes; markets.parquet, steps.parquet y manifest.json. Split por fecha: 8 682 mercados de train, 1 861 de val y 1 751 de test.
Entorno	**Implementado.** PolymarketEnv sobre Gymnasium: ejecución al primer precio después de 2 s, costos, liquidación e invariantes probadas con hypothesis.
Agentes	**Implementado.** Seis baselines, PPO y DQN (stable-baselines3), CQL y BC (d3rlpy) con una interfaz común; modelos finales con 5 semillas.
Evaluación y demo	**Implementado.** Backtest, métricas, prueba estadística, reportes de val y test, demo en Streamlit y análisis del comportamiento (EVA-12, PR #35).

Datos verificados con la API real

Familia elegida: **BTC Up or Down horario**, que resuelve con la vela de 1 hora de Binance; el spot de Binance reproduce el resultado. El precio por minuto sale del endpoint CLOB *prices-history*, que solo guarda 1 minuto durante 7 días, por eso el crudo se archivó a tiempo. Desde el 06/03/2026 estos mercados cobran comisión taker de **0,07 × p × (1 - p)** por acción.

Formulación del entorno

- **Episodio:** la vela de 60 minutos, con una decisión por minuto (60 pasos); liquidación a 1 o 0 al cierre, sin comisión.
- **Tiempo:** cada dato guarda cuándo ocurrió y cuándo estuvo disponible; el agente solo ve lo disponible hasta el instante de decisión.
- **Acciones:** WAIT, BUY_YES, BUY_NO y SELL; una posición de 20 USDC a la vez y ninguna apertura en los últimos 2 minutos. Una acción inválida se ejecuta como WAIT.
- **Costos:** medio spread de 0,01 más la comisión oficial; cada ejecución se contrasta con los trades reales (modo estricto).

> **Hallazgo que cambió el diseño:** con una grilla de 1 minuto, el precio guardado llega hasta 60 s tarde frente al spot y el agente podía "ver el futuro" (correlación de 0,90; alineando los tiempos baja a 0,22). Se resolvió con la ejecución al primer precio posterior a t + 2 s (ADR-017) y pruebas de no-fuga.

ORGANIZACIÓN Y PLAN DE TRABAJO

Equipo, fases y entregables

Cinco roles documentados; asignaciones tomadas de docs/03_roles.md al 02/11/2026.

Rol / integrante	Responsabilidad principal	Respaldo
R1 · Josué Líder técnico e integrador	Integración, contratos entre módulos, CLI, decisiones de diseño y coherencia de la documentación.	R4
R2 · Mathew Ingeniería de datos	APIs, dataset, features, split temporal y análisis exploratorio.	R5
R3 · Gustavo Entorno y recompensa	Simulador, cartera, costos, liquidación, invariantes y ablación de recompensa.	R4
R4 · Josué (desde el 02/11) Agentes RL	Baselines, PPO, DQN, dataset offline, CQL/BC y entrenamiento reproducible.	R3
R5 · Carlos Evaluación y demo	Backtesting, métricas, estadística, Streamlit, informe y presentación.	R2

Fases e hitos

Fase	Hito de cierre y estado
F0 · Preparación	APIs validadas, diseño decidido, crudo archivado y roles asignados. **Cerrada.**
F1 · Datos y entorno	Dataset v1, entorno validado y un baseline con episodios completos sobre datos reales. **Cerrada.**
F2 · Agentes online	PPO y DQN entrenados, prueba en entorno sintético y primera comparación en val. **Cerrada.**
F3 · Offline y ajuste	CQL/BC, ablación de la recompensa, hiperparámetros y modelos finales con 5 semillas. **Cerrada.**
F4 · Entrega final	Test evaluado una sola vez. **En curso:** faltan el informe, la presentación, el video y el release v1.0.

De 67 tareas, 46 están hechas contando EVA-12 (PR #35 abierto), 1 en curso, 1 omitida y 19 sin empezar, casi todas del documento. La CI corre en GitHub Actions: lint, pruebas en Python 3.11 y 3.12 y un smoke de entrenamiento.

Colaboración y criterio de finalización

El trabajo se concentra en la rama **proyecto**, con commits pequeños en formato convencional y pruebas por cada cambio de lógica. El 02/11, por tiempo, R1 mezcló los PR que esperaban revisión y lo dejó escrito en cada uno; lo que aparezca después irá en un PR de corrección. El uso de IA queda registrado: qué herramienta se usó, para qué y quién verificó el resultado.

METODOLOGÍA Y CRITERIOS DE ÉXITO

Evaluación y calidad

El protocolo busca comprobar qué aprende el agente y bajo qué supuestos funciona.

Comparación de estrategias

Los agentes se compararon con estrategias base: **aleatoria, no operar, favorito, momentum, reversión a la media y valor justo**. Todas pasan por el mismo entorno, costos y mercados. Los modelos se eligieron en val; test se usó una sola vez, con todo congelado.

- **Métrica principal:** D, la diferencia de PnL por mercado frente al mejor baseline (no operar), con un umbral de relevancia del 1 % del monto; además retorno, drawdown, operaciones y costos.
- **Repeticiones:** 5 semillas por agente final; IC por bootstrap de días y semillas, al 98,75 % por la corrección de Bonferroni.
- **Análisis:** ablación de la recompensa, modo estricto con trades reales y un análisis exploratorio de lo que aprendió cada agente (EVA-12).

Hipótesis y entregables evaluables

Se estudió si PPO supera al mejor baseline fuera de muestra, si penalizar el drawdown reduce el riesgo y si CQL iguala a las políticas que generaron sus datos. En test, PPO y DQN quedaron equivalentes a no operar, CQL inferior y BC peor sin diferencia relevante. **No superar los baselines es un resultado válido**, como se fijó antes.

Entregable	Verificación
Dataset y entorno	Reconstrucción con el mismo hash; check_env de Gymnasium; invariantes de cartera y saldo; pruebas de no-fuga. **Hecho.**
Agentes y comparación	Checkpoints de PPO, DQN, CQL y BC; reportes de val y de test con 5 semillas e IC. **Hecho.**
Demo e informe	Demo paso a paso y comparación de políticas **hechas**; falta el informe con resultados, ablación y limitaciones.
Calidad y reproducibilidad	367 pruebas aprobadas hoy; versiones fijadas en uv.lock; hash del crudo y del dataset; configuración, semilla y commit por corrida.

Pruebas

Con respuestas simuladas y semillas fijas. Incluyen un mercado sintético donde el precio es el valor justo, en el que ninguna política puede ganar, la comprobación de que alterar datos futuros no cambia la observación y el pipeline completo por la CLI.

Riesgos y límites del diseño

El riesgo previsto se confirmó: cerca del cierre el precio ya está calibrado y ninguna estrategia gana después de costos. PPO acertó el lado en el 99,76 % de sus compras a más de 0,98 en test, pero necesitaba el 99,88 % para no perder. El simulador es price-taker, aproxima **p(NO) como 1 - p(YES)** y solo la mitad de las ejecuciones de PPO en test tiene un trade real que la respalde.

CAMBIOS REGISTRADOS · 2 DE NOVIEMBRE DE 2026

Avances y estado actual

Resumen basado en el historial de Git, los PR, los archivos del proyecto y las pruebas ejecutadas.

Mezcla en proyecto	Cambio
01/11 19:59 · #6	Modelo de costos con medio spread y comisión taker.
02/11 06:27 · #1 a #10	Contratos de datos, klines de Binance, split temporal, dataset, EDA, trades, features y protocolo de evaluación.
02/11 06:31 · #11 a #29	Entorno, baselines, backtest, validación con trades, métricas, entrenamiento de PPO, DQN, CQL y BC, reportes, demo y CI.
02/11 07:54 · #30 a #33	Ablación de la recompensa, umbrales de los baselines, hiperparámetros, modelos finales y reporte de val.
02/11 09:58 · #34	Evaluación final en test, una sola vez.
02/11 17:47 · #35 (abierto)	Análisis del comportamiento de los agentes (EVA-12), con la CI en verde.

Implementado y comprobado

- **Resultado en test:** con no operar como mejor baseline, la media de D fue -0,0005 para PPO (IC de -0,0029 a +0,0018), -0,0018 para DQN, -0,0126 para CQL y -0,0014 para BC. CQL parecía el mejor en val (+0,0109).
- **Qué aprendió PPO:** compra el favorito cerca del final (minuto mediano 54) y decide sobre todo con la distancia de BTC a la apertura. Cada acierto deja unos 0,02 USDC y un error cuesta 20: en test, -27,23 USDC netos en esas compras.

> **Verificación local:** `uv run pytest -q`
> **Resultado: 367 pruebas aprobadas** y 10 lentas o de red excluidas por defecto. La CI del PR #35 pasó lint, pruebas en 3.11 y 3.12 y el smoke de entrenamiento.

Pendiente frente al plan

Revisar y mezclar el PR #35; mostrar en la demo el mejor agente en un mercado de test (EVA-08); escribir el documento (DOC-02 a DOC-09), las slides, la presentación y el video; publicar el release v1.0 (INF-09), la reproducción por un tercero (INF-10) y pasar proyecto a main.

Fuentes: **README.md**, **docs/** (tablero en docs/04_tareas.md), **reports/val/**, **reports/test/**, **src/polym4v/**, **tests/** y el historial de Git.

> **Repositorio del proyecto · rama proyecto**
> https://github.com/donmatthiuz/RL/tree/proyecto